San Francisco State University

Software Engineering CSC 648 - 848

Project - Ascenda

Section 06 Team 1

Alisha Sinha	Team Lead
Ian Brown	Scrum Master
Zaw Htut	Git Master
Prashrit Magar	AI Master
Geo Wong	Back-end Lead
Dong Wu Yu	Front-end Lead

Milestone 2

October 20th, 2026

Prototype URL:
https://frontend-phi-hazel-cmbz8sepjp.vercel.app/

GitHub Repository:
https://github.com/SFSU-CSC648-TeamOne/SFSU-csc648-06-Fall26-team01

1. Data Definitions V2

1.1 Overview

Ascenda is an adaptive fitness-training web application designed to help users find appropriate exercises, follow structured training programs, manage workout time, and track fitness progress. The system organizes information related to user profiles, fitness assessments, exercise catalogs, training programs, workout sessions, exercise modifications, and workout history.

The primary data storage system is PostgreSQL, accessed through SQLAlchemy ORM models in the Python FastAPI backend. The frontend uses Next.js to display exercise and workout information and interact with backend services.

This section expands the Milestone 1 data definitions by identifying the major data entities, their attributes, relationships, and purposes. It preserves the terminology established in Milestone 1 while distinguishing between implemented data models and planned application features.

1.2 Users and Personal Fitness Information

1.2.1 User

A User is an individual who uses Ascenda to browse exercises, follow workout programs, and maintain personal workout information.

Database table: users

Attribute	Data Type	Description
id	Integer	Unique identifier for the user
email	String	Email address used for registration and authentication
password_hash	String	Hashed representation of the user's password
role	String	Role assigned to the user

Relationships: A User may have one User Profile, multiple Fitness Assessments, and multiple Workout Records.

User privileges: Milestone 1 specifies that regular Users should access their own profiles, workout history, assessments, and preferences. Administrators are intended to maintain shared

application data such as exercise catalogs and equipment definitions, without unrestricted access to personal workout histories.

The backend currently implements user registration, login, and authenticated access to user-specific workout records. The complete administrator permission system has not been verified.

Constraints requiring verification: Exact email length, password rules, allowed role values, and database constraints.

1.2.2 User Profile

A User Profile contains information used to describe the user's fitness background, objectives, and training preferences.

Database table: user_profiles

Attribute	Data Type	Description
user_id	Integer	Identifier of the associated User
goals	List	User's fitness objectives
experience_level	String	User's fitness experience classification
days_per_week	Integer	Preferred number of weekly workout days
session_minutes	Integer	Preferred workout duration in minutes

Milestone 1 also identifies workout location, physical capability constraints, and equipment availability as important profile concepts. Equipment availability is represented through the user_equipment association table.

Relationships: Each User Profile belongs to a User and may be associated with multiple Equipment items.

Implementation status: The UserProfile ORM model exists. A complete frontend profile editing workflow and corresponding update endpoint have not been verified.

1.2.3 Fitness Goal

A Fitness Goal describes the training outcome a user wants to achieve. Milestone 1 identifies goals such as strength, cardiovascular fitness, muscular endurance, flexibility, mobility, general health, appearance, and sports performance.

Data representation: The goals attribute in UserProfile and the goal attribute in TrainingProgram represent fitness goals.

Implementation status: Fitness goals are represented in existing data models. A separate fitness_goals database table has not been identified in the available architecture documentation.

Verification needed: The complete list of supported goal values and their validation rules.

1.2.4 Fitness Assessment

A Fitness Assessment is a structured evaluation intended to establish a user's starting fitness level and provide a measurable reference for evaluating progress over time.

Database table: assessments

Attribute	Data Type	Description
user_id	Integer	Identifier of the User completing the assessment
answers	Dictionary	Collection of responses provided during the assessment
score	Integer	Calculated assessment score
experience_level	String	Experience classification associated with the assessment

Relationships: One User may have multiple Fitness Assessments, allowing results from different points in time to be retained.

Implementation status: The Assessment ORM model exists. Automated assessment scoring, assessment submission, and user-facing assessment screens have not been verified as implemented.

Additional definitions required: Exact assessment questions, response formats, score ranges, and classification rules remain undefined in the supplied documentation.

1.2.5 Physical Capability Constraint

A Physical Capability Constraint describes a limitation identified by a user that may affect exercise selection or appropriate workout modifications.

Milestone 1 includes physical limitations and changes in mobility or training ability as important considerations.

This information is intended to help Ascenda identify suitable exercise modifications or alternatives. It is not intended to represent a medical diagnosis.

Implementation status: This is a defined Milestone 1 concept. A separate database table or complete set of corresponding ORM attributes has not been verified.

1.3 Exercise Catalog Data

1.3.1 Exercise

An Exercise represents an individual physical activity or movement available within Ascenda. Exercises provide the basic building blocks of workout sessions.

Database table: exercises

Attribute	Data Type	Description
id	Integer	Unique identifier of the Exercise
wger_id	Integer	Identifier associated with imported wger exercise information
name	String	Name of the Exercise
description	String	Exercise description and supporting information
default_role	String	Default classification of the Exercise within workout planning
default_minutes	Integer	Default estimated duration in minutes

The architecture also identifies exercise difficulty, equipment associations, muscle groups, and source attribution as part of the exercise catalog. Their complete column definitions are not contained in the supplied documentation.

Relationships:

- An Exercise may be associated with multiple Muscle Groups.
- An Exercise may be associated with multiple Equipment items.
- An Exercise may appear in multiple Workout Sessions.
- An Exercise may have associated 3D visualization metadata.
- An Exercise may be referenced by multiple Performance Entries.

Implementation status: Backend exercise listing, filtering, and detail retrieval are implemented in source. Catalog data can be populated through the repository's wger import script.

1.3.2 Muscle Group

A Muscle Group identifies an area of the body targeted by an exercise.

Database table: muscle_groups

Attribute	Data Type	Description
id	Integer	Unique identifier of the Muscle Group
name	String	Name of the Muscle Group

Relationships: Exercises and Muscle Groups have a many-to-many relationship represented through exercise_muscles. The association model also includes an is_primary attribute to identify primary muscle relationships.

Implementation status: Muscle group retrieval is supported through the backend API.

1.3.3 Equipment

Equipment represents an item or category of training equipment used or associated with an exercise.

Database table: equipment

Attribute	Data Type	Description
id	Integer	Unique identifier of the Equipment
name	String	Name of the Equipment

Milestone 1 identifies dumbbells, barbells, resistance bands, cable machines, benches, and cardio machines as examples of exercise equipment.

Relationships:

- Equipment may be associated with multiple Exercises through exercise_equipment.
- Equipment available to a user may be represented through user_equipment.

Implementation status: Equipment retrieval and catalog relationships are represented in the backend.

1.3.4 Core Exercise and Accessory Exercise

Ascenda distinguishes exercises according to their importance within a workout session.

A **Core Exercise** is central to the primary purpose of a workout. It is intended to be preserved when a workout needs to be shortened.

An **Accessory Exercise** provides additional training benefits but has lower priority than the session's core exercises. Accessory exercises are the preferred candidates for removal when workout time becomes limited.

Data representation:

Attribute	Model	Description
default_role	Exercise	Default exercise classification
role	SessionExercise	Classification within a particular Workout Session

The documented architecture identifies core and accessory role classifications. A session-specific exercise role can differ from the exercise's default role.

Implementation status: These classification fields exist in the backend models. Automatic exercise removal or workout adaptation based on these classifications has not been verified as implemented.

1.3.5 Exercise Modification

An Exercise Modification is an adjusted version of an exercise intended to accommodate a different difficulty level or physical capability.

Examples discussed in the Milestone 1 concept include selecting easier or harder exercise variations according to a user's capabilities.

Implementation status: Exercise Modification is defined conceptually in Milestone 1. A dedicated Exercise Modification table, relationship schema, or automated modification service has not been identified in the reviewed implementation.

1.3.6 Exercise Substitution

An Exercise Substitution is an alternative exercise selected when the originally prescribed exercise is unsuitable or unavailable.

Milestone 1 identifies equipment availability, physical capability, and changing workout conditions as reasons a substitution may be needed.

Implementation status: Exercise substitution is a planned application concept. The Performance Entry data area includes substitution details according to the architecture overview, but a complete substitution schema or automated substitution algorithm has not been verified.

1.4 Training Programs and Workout Sessions

1.4.1 Training Program

A Training Program represents an organized collection of Workout Sessions designed around a fitness goal.

Database table: training_programs

Attribute	Data Type	Description
id	Integer	Unique identifier of the Training Program
name	String	Name of the Training Program
goal	String	Fitness goal associated with the program
length_weeks	Integer	Expected program duration in weeks

Relationships: One Training Program may contain multiple Workout Sessions.

Implementation status: Program listing and filtering are implemented in the backend. Program recommendations based on user assessment results remain a planned feature.

1.4.2 Workout Session

A Workout Session represents one planned workout within a Training Program.

Database table: workout_sessions

Attribute	Data Type	Description

id	Integer	Unique identifier of the Workout Session
program_id	Integer	Identifier of the associated Training Program
name	String	Name of the Workout Session
position	Integer	Ordering of the session within its program

Relationships:

- Each Workout Session is associated with a Training Program.
- Each Workout Session may contain multiple SessionExercise records.
- A Workout Session may be referenced by Workout Records.

Implementation status: Workout Session retrieval, detail display, and filtering are represented in the current frontend/backend implementation.

1.4.3 Session Exercise

A SessionExercise represents an individual prescribed exercise within a Workout Session. It connects an Exercise to its prescribed workload.

Database table: session_exercises

Attribute	Data Type	Description
workout_session_id	Integer	Identifier of the parent Workout Session
exercise_id	Integer	Identifier of the prescribed Exercise
role	String	Core or accessory classification within the session
sets	Integer	Number of prescribed sets
reps	Integer	Number of prescribed repetitions
minutes	Integer	Estimated time assigned to the exercise

Relationships: SessionExercise connects WorkoutSession and Exercise.

Purpose: These attributes support ordered workout presentation, prescribed workloads, expected duration calculations, and the data foundation for future adaptation features.

1.4.4 Workout Duration

Workout Duration represents the estimated time required to complete a Workout Session.

The architecture documents exercise-level time values and calculated workout-session durations.

Relevant data items:

- Exercise.default_minutes — default estimated exercise duration.
- SessionExercise.minutes — duration allocated to an exercise within a session.
- UserProfile.session_minutes — user's preferred session duration.
- WorkoutRecord.available_minutes — available time associated with a recorded workout.

Implementation status: Workout duration calculation and filtering by maximum minutes are represented in the backend and /workouts frontend page.

Automatically restructuring an existing session to fit a shorter duration remains a proposed enhancement.

1.4.5 Availability

Availability describes the amount of time a user has for a workout on a particular occasion.

Unlike preferred workout duration, Availability may change from one workout to another.

Data representation: Available time is represented in the WorkoutRecord model through available_minutes. The workout-session search API also accepts maximum-duration filtering.

Implementation status: Available-time filtering exists. Automatic adjustment of exercise prescriptions to fit a user's available time has not been verified.

1.4.6 Program Customization

Program Customization represents a persistent change to a user's training program.

Milestone 1 distinguishes persistent program customization from a temporary Exercise Substitution performed during one workout.

Planned customization may involve workout frequency, difficulty, exercise selections, or scheduling.

Implementation status: Program Customization is defined in Milestone 1, but no separate persistence model or complete customization workflow has been identified in the current architecture review.

1.5 Workout Performance and Progress Tracking

1.5.1 Workout Record

A Workout Record stores information about a workout performed by a user.

Database table: workout_records

Attribute	Data Type	Description
id	Integer	Unique identifier of the Workout Record
user_id	Integer	Identifier of the User who owns the record
performed_at	DateTime	Date and time of the recorded workout
available_minutes	Integer	Available workout time associated with the record

The architecture documentation also identifies workout timing and notes within the workout-record data area, but does not establish their exact attribute definitions.

Relationships:

- Each Workout Record belongs to a User.
- A Workout Record may reference a Workout Session.
- A Workout Record may contain multiple Performance Entries.

Implementation status: Authenticated record creation and user-specific record retrieval are implemented in backend source. Complete frontend workout logging integration has not been verified.

1.5.2 Performance Entry

A Performance Entry stores information about an individual exercise performed during a workout.

Database table: performance_entries

Attribute	Data Type	Description
id	Integer	Unique identifier of the Performance Entry
exercise_id	Integer	Identifier of the performed Exercise

reps	Integer	Number of repetitions recorded
weight_kg	Float	Recorded exercise weight in kilograms
status	String	Status of the performance entry

The architecture also identifies sets, duration, and exercise substitution details as part of performance tracking. Exact additional attributes require verification.

Relationships:

- Each Performance Entry belongs to a Workout Record.
- Each Performance Entry references an Exercise.

Implementation status: Performance entries are represented in backend models and workout-record creation services.

1.5.3 Workout History

Workout History represents the collection of a user's past Workout Records.

It allows users to review previous workout activity and recorded performance.

Data representation: Workout History is derived from WorkoutRecord and related PerformanceEntry records rather than an independently identified workout_history database table.

Implementation status: Authenticated workout history retrieval and record detail endpoints exist in the backend. A connected user-facing history screen has not been verified.

1.5.4 Program Progress

Program Progress describes the user's overall advancement through a Training Program.

Milestone 1 identifies completed sessions, assessment results, milestones, and training consistency as relevant information for determining progress.

Data representation: Existing Workout Records, Performance Entries, Fitness Assessments, and Training Program relationships provide a partial data foundation for progress evaluation.

Implementation status: Program Progress remains a conceptual feature. A dedicated progress calculation service or independent ProgramProgress model has not been verified.

1.6 Exercise Media and Supporting Data

1.6.1 Exercise Model Metadata

Exercise Model Metadata identifies multimedia resources associated with an Exercise.

Database table: exercise_models

Attribute	Data Type	Description
exercise_id	Integer	Identifier of the associated Exercise
model_url	String	URL of an associated 3D model
video_url	String	URL of an associated demonstration video

The architecture overview also identifies optional animation-name and attribution information.

Relationships: An Exercise may have an associated ExerciseModel metadata record.

Implementation status: The backend supports listing and retrieving exercise-model metadata.

1.6.2 Exercise Demonstration Media

Ascenda includes multimedia resources intended to assist users in understanding exercises.

Data Item	Documented Format	Usage
3D exercise model	GLB	Interactive 3D visualization
Exercise demonstration video	MP4	Playback of exercise demonstration
Model/video location	URL string	References to associated media resources
Media attribution	Text metadata	Source and credit information where stored

The current frontend uses renders the Scaniverse-exported GLB with React Three Fiber and Drei, and maps a selected local MP4 onto a planar mesh (2-Dimensional slice) inside the 3D model via VideoPlane.tsx and Three.js VideoTexture. The video is muted and loops. Users can switch between two hard-coded demonstrations and adjust the video position. This spatially

placed 2D video is a proof-of-concept, our ultimate goal is a 3D video of roughly 180 degrees viewability (with respect to the privacy of the person performed the demonstration in the video).

We plan for our video demonstrations to be attached to relevant exercise within our database. And for these demonstration to be accessed via a button beside a prescribed exercise within an exercise program in our frontend.

The available documentation does not specify maximum file sizes, image dimensions, video resolution, encoding settings, or duration limits. These technical constraints must be confirmed from the actual assets and application configuration before final acceptance.

The exercise demonstration system does not require a live Niantic API call in the documented implementation.

1.7 Database Relationships

The database is organized around the following main relationships.

Primary Entity	Related Entity	Relationship
User	UserProfile	One-to-zero-or-one
User	Assessment	One-to-many
User	WorkoutRecord	One-to-many
UserProfile	Equipment	Many-to-many through user_equipment
Exercise	Equipment	Many-to-many through exercise_equipment
Exercise	MuscleGroup	Many-to-many through exercise_muscles
TrainingProgram	WorkoutSession	One-to-many
WorkoutSession	SessionExercise	One-to-many
SessionExercise	Exercise	Many-to-one
WorkoutRecord	WorkoutSession	Optional session reference
WorkoutRecord	PerformanceEntry	One-to-many
PerformanceEntry	Exercise	Many-to-one
Exercise	ExerciseModel	One-to-zero-or-one

These relationships reflect the documented high-level SQLAlchemy ORM model. Additional foreign keys, constraints, and association attributes are omitted from this overview.

1.8 Data Access and User Privileges

Ascenda distinguishes personal fitness information from shared exercise and workout catalog data.

Data Category	Regular User Access	Administrator Access
Personal profile	Access own profile; editing is planned	No unrestricted access established
Fitness assessments	Access own assessments is planned	No unrestricted access established
Exercise catalog	Browse and search	Catalog maintenance intended in M1
Equipment catalog	View equipment data	Catalog maintenance intended in M1
Training programs	Browse available programs	Maintenance permissions not verified
Workout records	Create and retrieve own records through authenticated backend APIs	No unrestricted access established
Workout history	Retrieve own history through authenticated backend APIs	No unrestricted access established

Implementation distinction: This table combines Milestone 1's intended access policy with backend operations confirmed by the architecture review. It does not assert that a complete role-based access-control system or all listed user interfaces have been implemented.

The backend uses password hashing and bearer-token authentication for user-specific operations. Public catalog endpoints support information retrieval without the workout-record ownership requirements.

1.9 Data Types, Validation, and Storage Requirements

The following data types are explicitly represented in the provided UML documentation.

Data Type	Usage
Integer	Entity identifiers, sets, repetitions, duration, assessment scores, program length
String	Names, descriptions, roles, goals, and status values
Float	Recorded exercise weight
DateTime	Workout performance timestamps
List	User fitness goals
Dictionary	Fitness assessment answers
URL represented as String	Exercise model and video references

Detailed database column sizes, nullability rules, unique constraints, indexes, and check constraints are defined by the actual ORM source and are not completely enumerated in the supplied architecture summaries.

The existing documentation does not establish fixed maximum media sizes or the final structure of complex assessment answers.

These properties should be documented using verified model declarations and approved validation rules rather than assumed values.

1.10 Milestone 1 to Milestone 2 Data Refinement

Milestone 1 established the conceptual data model for Ascenda. Milestone 2 refines this model by identifying the database entities and implementation attributes currently represented in the backend.

Milestone 1 Concept	Milestone 2 Representation
User	users / User
Administrator	Intended role represented conceptually; User.role exists
User Profile	user_profiles / UserProfile
Fitness Goal	UserProfile goals and TrainingProgram goal

Fitness Assessment	assessments / Assessment
Training Program	training_programs / TrainingProgram
Workout Session	workout_sessions / WorkoutSession
Exercise	exercises / Exercise
Core / Accessory Exercise	Exercise.default_role and SessionExercise.role
Equipment	equipment / Equipment
Exercise Modification	Concept defined; implementation model not verified
Exercise Substitution	Concept defined; full implementation model not verified
Program Customization	Concept defined; persistent customization model not verified
Physical Capability Constraint	Concept defined; exact storage structure not verified
Workout Record	workout_records / WorkoutRecord
Performance Entry	performance_entries / PerformanceEntry
Workout History	Derived from WorkoutRecord and PerformanceEntry
Program Progress	Concept defined; calculation model not verified
Availability	WorkoutRecord.available_minutes and duration filtering
Gym / Training Zone	Concept defined in M1; separate ORM implementation not verified

This refinement preserves the original fitness-related terminology while providing greater detail about how the current backend represents supported data.

Clarification for sources: The above table entries correspond to SQLAlchemy declarations in app/backend/app/models (users.py, catalog.py, tracking.py, and media.py). They describe stored structure and do not establish implemented assessment scoring, profile editing, or future upload validation.

Data entity	**Additional source-defined fields / constraints**

user	display_name (optional; 100 character), created_at; email unique (255 characters), role constrained to UserRole values
user_profiles	id, physical_constraints (optional text), updated_at, one profile per user (unique user_id); goals stored as JSON
assessments	id, created_at; answers JSON, score integer, experience_level constrained to ExperienceLevel values
exercises	Category, difficulty, source, license_name, license_author; equipment and primary/secondary muscle mappings
training_programs	Experience_level and optional description in addition to name, goal, length_weeks
workout_sessions / session_exercises	session guidance; prescribed item id and position: unique workout_session_id + position; sets, reps and minutes nullable
workout_records / performance_entries	Workout_session_id (optional), duration_minutes, notes, entry workout_record_id + position; sets, reps and minutes nullable
exercise_models	id, animation_name (optional), credit (optional); at most one model-metadata record per exercise.

1.11 Outstanding Verification Requirements

The following details remain to be confirmed against the complete source code and approved specifications:

1. Exact SQLAlchemy column definitions, maximum string lengths, nullability, primary keys, foreign keys, and validation constraints.
2. Supported User role values and the final administrator permission structure.
3. Fitness Assessment answer schema, scoring method, and supported experience-level values.
4. Complete fields for Exercise difficulty, provenance, exercise relationships, and performance tracking.
5. The persistence design for Exercise Modification, Exercise Substitution, and Program Customization.

6. Maximum file size, resolution, duration, and validation requirements for GLB and MP4 assets.
7. Production database migration status and verification of populated records.

These are verification items rather than confirmed implementation details. Any values not yet specified must be finalized through the backend source definitions and team-approved requirements.

2. Functional Requirements V2

2.1 Overview

This section expands the functional requirements defined in Milestone 1. Each requirement is divided into smaller, testable functions while keeping the original requirement IDs (R1–R13).

The priorities are based on user interviews, application usability, Ascenda's main fitness features, and development feasibility. Priority 1 focuses on the essential features needed for a usable fitness application. Priority 2 improves personalization and user experience. Priority 3 covers advanced functions that can be added if time allows.

Priority Definitions

- **Priority 1 – Must Have:** Required for the main application.
- **Priority 2 – Desired:** Important improvements but not required for the basic application.
- **Priority 3 – Opportunistic:** Additional features depending on available time and resources.

2.2 Account Management

R1. Registration and Login

M1 Priority: 1

Users must be able to create an account and log in to manage personal fitness data.

ID	Functional Requirement	Priority
R1.1	The system shall allow new users to register using an email address and password.	1
R1.2	The system shall prevent registration using an email address already associated with an account.	1
R1.3	The system shall allow registered users to log in using valid credentials.	1
R1.4	The system shall reject invalid login credentials.	1
R1.5	The system shall store passwords as hashes instead of plain text.	1
R1.6	The system shall restrict personal workout records to authenticated users and their own accounts.	1

R2. User Profile

M1 Priority: 1

The user profile stores fitness preferences that can help with workout selection.

ID	Functional Requirement	Priority
R2.1	The system shall allow users to provide fitness goals.	1
R2.2	The system shall store the user's experience level.	1
R2.3	The system shall allow users to specify available exercise equipment.	1
R2.4	The system shall store preferred workout duration and training days per week.	1
R2.5	The system should allow users to update their fitness preferences.	2
R2.6	The system should allow users to provide physical capability constraints for future exercise selection.	2

2.3 Exercises and Workout Programs

R3. Exercises

M1 Priority: 1

The system must store exercise information and provide guidance so users can understand their workouts.

ID	Functional Requirement	Priority
R3.1	The system shall store the name and description of each exercise.	1
R3.2	The system shall associate exercises with their target muscle groups.	1
R3.3	The system shall associate exercises with required or related equipment.	1
R3.4	The system shall provide exercise instructions or available demonstration media.	1
R3.5	The system shall classify prescribed exercises as core or accessory exercises.	1
R3.6	The system should display available 3D exercise models.	2

R4. Fitness Assessment

M1 Priority: 1

The fitness assessment helps users establish their initial experience level and provides a starting point for future training.

ID	Functional Requirement	Priority
R4.1	The system shall provide an initial fitness assessment for users.	1
R4.2	The system shall collect answers to the assessment questions.	1
R4.3	The system shall calculate an assessment result using defined scoring rules.	1
R4.4	The system shall assign an experience level based on the assessment result.	1
R4.5	The system shall store assessment answers and results for future reference.	1
R4.6	The system should allow users to repeat the assessment and compare results.	2

Specification dependency: The team must finalize the assessment questions, scoring rules, and experience-level thresholds. These details are not defined in the supplied M1 or architecture documents.

R5. Exercise Catalog

M1 Priority: 1

Users must be able to browse exercises and available training programs.

ID	Functional Requirement	Priority
R5.1	The system shall display available exercises from the database.	1
R5.2	The system shall allow users to search exercises by name or keywords.	1
R5.3	The system shall allow users to filter exercises by muscle group, equipment, and difficulty.	1
R5.4	The system shall display available training programs and workout sessions.	1
R5.5	The system shall display exercises, sets, and repetitions within a selected workout session.	1
R5.6	The system should allow users to filter training programs by goal and program length.	2

R6. Workout Duration

M1 Priority: 1

Users must be able to see expected workout durations. Workout adaptation is also an important part of Ascenda's goal of supporting users with limited time.

ID	Functional Requirement	Priority
R6.1	The system shall store estimated durations for exercises.	1
R6.2	The system shall calculate the expected duration of a workout session from its prescribed exercises.	1
R6.3	The system shall display expected workout duration before the user begins a session.	1
R6.4	The system shall allow users to find workout sessions that fit a selected maximum duration.	1
R6.5	The system should allow users to enter their available workout time and request a shortened session.	2

R6.6	The system should prioritize keeping core exercises and removing accessory exercises when shortening a session.	2
R6.7	The system should notify users when the remaining core exercises cannot fit the selected time.	2

R7. Program Recommendation (Using AI)

M1 Priority: 2–3

The system should help users select training programs based on their personal fitness information.

ID	Functional Requirement	Priority
R7.1	The system should identify programs matching a user's fitness goals.	2
R7.2	The system should consider experience level when selecting suitable programs.	2
R7.3	The system should consider available equipment and preferred workout duration.	2
R7.4	The system may use AI-assisted recommendations to suggest personalized training programs.	3

2.4 Workout Logging and Progress

R8. Workout Logging

M1 Priority: 1–2

Users should be able to record the exercises and performance information from completed workouts.

ID	Functional Requirement	Priority
R8.1	The system shall allow authenticated users to create workout records.	1
R8.2	The system shall allow users to record completed exercises and repetitions.	1

R8.3	The system shall associate each workout record with its owner.	1
R8.4	The system should allow users to record sets, weight, duration, and completion status when applicable.	2
R8.5	The system should support recording exercises that were skipped or substituted.	2
R8.6	The system should keep workout logging optional when users only want to browse or follow workouts.	2

R9. Workout History

M1 Priority: 1–2

Users must be able to retrieve their saved workout information.

ID	Functional Requirement	Priority
R9.1	The system shall retrieve workout records belonging to the authenticated user.	1
R9.2	The system shall allow users to view details of a selected workout record.	1
R9.3	The system should display previous performance information for completed exercises.	2
R9.4	The system should allow users to compare performance information from different workouts.	2

R10. Workout Tracking

M1 Priority: 2–3

Workout tracking helps users understand how their training performance changes over time.

ID	Functional Requirement	Priority
R10.1	The system should use stored workout records to summarize previous training activity.	2

R10.2	The system should display changes in recorded repetitions or weights across workouts.	2
R10.3	The system may display performance trends using charts.	3
R10.4	The system may show program completion progress and training milestones.	3

R11. Workout Adjustment

M1 Priority: 2–3

Workout adjustment allows users to modify training according to equipment availability, physical capability, and previous performance.

ID	Functional Requirement	Priority
R11.1	The system should allow users to select an alternative exercise when required equipment is unavailable.	2
R11.2	The system should support exercise variations for different experience or ability levels.	2
R11.3	The system should allow users to adjust workout difficulty where supported.	2
R11.4	The system may use assessment results and workout history to suggest future workout adjustments.	3
R11.5	The system may support persistent program customization, including changes to difficulty or workout frequency.	3

2.5 Gym and Equipment Features

R12. Gym Scanning

M1 Priority: 2–3

Gym scanning is an additional feature intended to help users identify equipment in a training environment.

ID	Functional Requirement	Priority
R12.1	The system may display available 3D representations of a training environment.	3
R12.2	The system may support identifying equipment in a scanned training environment.	3

R13. Equipment Details

M1 Priority: 2–3

Equipment details can help users identify exercises that match available equipment.

ID	Functional Requirement	Priority
R13.1	The system may display exercise information associated with equipment identified in a scanned environment.	3
R13.2	The system may suggest workouts that use the identified equipment.	3

2.6 Priority Summary and Justification

Requirements are prioritized based on the main user problems described in Milestone 1, development feasibility, and the need for a complete application.

Priority	Requirements	Justification
1 – Must Have	Registration, basic profiles, exercise information, fitness assessment, exercise catalog, workout duration display/filtering, basic logging and history	Supports Ascenda's main workout selection and tracking functions.
2 – Desired	Profile editing, 3D demonstrations, repeated assessments, workout shortening, program matching, detailed logging, substitutions, and workout adjustments	Improves personalization, convenience, and user retention.

3 – Opportunistic	AI recommendations, advanced progress visualizations, persistent program customization, gym scanning, and scan-based equipment recommendations	Adds advanced features but requires more development and integration work.

Priority Justification

User needs and usability: Milestone 1 interviews showed that users need clear exercise guidance, structured workouts, and flexibility with limited time. Therefore, the exercise catalog, workout sessions, and duration filtering receive Priority 1.

Marketing and competitive value: Ascenda's main differences are fitness assessments, time-aware workouts, and workout adaptability. The initial assessment receives Priority 1, while advanced workout adaptation receives Priority 2 because additional algorithms and testing are needed.

Business and development feasibility: The team has a limited development schedule. Existing backend functions for authentication, exercise data, workout sessions, and workout records provide a foundation for the main application. Advanced AI and gym-scanning features have lower priority to reduce implementation risk.

Commitment for Milestone 3: Priority 1 requirements represent the proposed must-have scope for the next development milestone. The team should confirm these priorities before submission, especially assessment scoring and frontend integration, since their complete implementation has not yet been verified.

3. UI Mockups and UX Flows

3.1 Overview

This section presents six high-priority Ascenda user flows. The black-and-white wireframes show the main screens, inputs, and navigation. They are design mockups, not evidence that every screen is implemented. The team can refine these layouts as the UI develops.

Wireframe status: The workout-session list and detailed API integration is present in the reviewed code. Authentication, assessment, workout adaptation, and workout logging screens below are proposed until frontend integration is verified.

3.2 Registration and Login

Requirements: R1. A user creates an account or signs in to access personal workout information.

ASCENDA | Register	ASCENDA | Login
Email: [_____]	Email: [_____]
Password: [_____]	Password: [_____]
[Create Account]	**[Log In]**
Already registered? Log in	Invalid credentials: show an error

Figure 3.1: Register | *Figure 3.2: Login*

UX flow: Open account page → Register or log in → Validate credentials → Open account area

Implementation note: Proposed UI; backend registration and login endpoints exist.

3.3 Profile and Initial Assessment

Requirements: R2, R4. A new user enters training preferences and completes an initial assessment.

ASCENDA | Profile Setup	ASCENDA | Assessment Result
Fitness goal: [Select goal v]	Assessment questions
Experience: [Select level v]	**[Answer 1] [Answer 2]**
Equipment: [Select items]	**[Submit Assessment]**
Days/week: [ ] Minutes: [ ]	Result: [Score / Level]
[Continue to Assessment]	**[Continue]**

Figure 3.3: Profile Setup | *Figure 3.4: Assessment Result*

UX flow: Enter profile preferences → Answer assessment → Submit answers → View score and level

Implementation note: Proposed UI and scoring flow; profile and assessment ORM models exist.

3.4 Exercise Catalog

Requirements: R3, R5. A visitor finds exercises using keywords and filters, then reads exercise details.

ASCENDA | Exercise Search	ASCENDA | Exercise Details
Search: [_____]	Exercise: [Name]
Muscle: [v] Equipment: [v]	Muscle: [Group]
Difficulty: [v]	Equipment: [Item]
[Search]	Instructions: [Description]
Result A [View]	**[Demonstration / Media]**
Result B [View]	**[Back to results]**

Figure 3.5: Exercise Search | *Figure 3.6: Exercise Details*

UX flow: Open catalog → Search / filter → Select exercise → View details

Implementation note: Exercise lookup APIs exist; homepage search remains illustrative and not fully connected.

3.5 Workout Selection

Requirements: R5, R6. A visitor filters sessions by difficulty, muscle group, and maximum time.

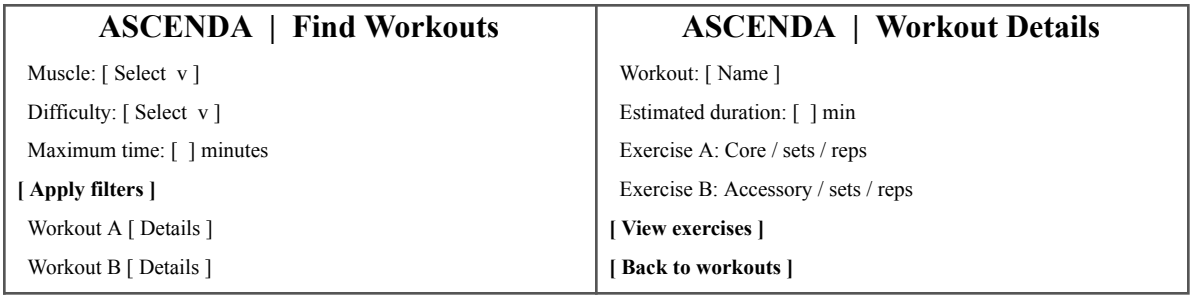


Figure 3.7: Find Workouts

Figure 3.8: Workout Details

UX flow: Open workouts → Apply filters → Choose a session → Read exercises and duration

Implementation note: The /workouts frontend/API listing and detail path exists; live data must be checked.

3.6 Workout Duration Adjustment

Requirements: R6. A user requests a shorter session while keeping core exercises when possible.

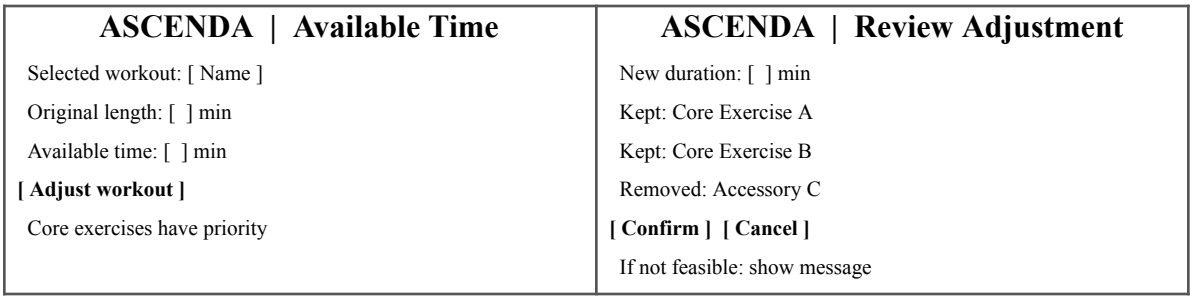


Figure 3.9: Available Time

Figure 3.10: Review Adjustment

UX flow: Select workout → Enter available time → Review proposed changes → Confirm or choose another session

Implementation note: Proposed advanced flow; filtering by maximum duration exists, but automatic adjustment is not verified.

3.7 Workout Logging and History

Requirements: R8, R9. An authenticated user records performed exercises and reviews previous workouts.

ASCENDA | Record Workout	**ASCENDA | Workout History**
Workout: [Name]	My Workout History
Exercise: [Name]	**[Date] Workout A [View]**
Sets: [ ] Reps: [ ]	**[Date] Workout B [View]**
Weight (kg): [ ]	Selected record:
Status: [Select v]	Exercises / reps / weight
[Save record]	**[Back]**

Figure 3.11: Record Workout | Figure 3.12: Workout History

UX flow: Complete workout → Enter performance → Save record → View user-owned history

Implementation note: Backend workout-record creation and retrieval exist; the frontend screens are proposed.

3.8 Evaluation of UX Principles

The following table describes how the proposed designs address the six UX principles. Actual usability and accessibility still need to be checked through testing.

Principle	How the design addresses it
Useful	Core tasks support exercise search, workout selection, assessment, and workout logging.
Usable	Screens use short forms, labeled filters, and clear next actions.
Desirable	The wireframes keep only essential fitness information on each screen.
Accessible	Controls are labeled and intended to support keyboard access; implementation testing is still needed.
Findable	Users can locate exercises and workouts using search, filters, and visible navigation.

Credible	Exercise details, estimated durations, and saved records are shown with clear labels; results must come from actual stored data.

4. High level Architecture, Database Organization

4.1 System Architecture

Ascenda uses a three-tier architecture: Next.js/React frontend, FastAPI backend deployed as Vercel project. The backend reads and writes a PostgreSQL database hosted on Supabase through SQLAlchemy. Browser sends requests to /api/* then are forwarded by Next.js to edit the backend. The frontend sends REST requests through a Next.js /api rewrite to the backend. The /workouts page uses the workout-session API; the homepage still includes hardcoded exercise examples.

Data types used from wger's api:

- Exercise - muscle group, equipment, difficulty
- Workout Session - muscle, difficulty, duration

Database is hosted on Supabase and backend endpoint calls are set up as:

- Routes → Services → Repositories → Models

wger api is a one-time call as it does not need to be called live when a user is browsing the exercise catalog.

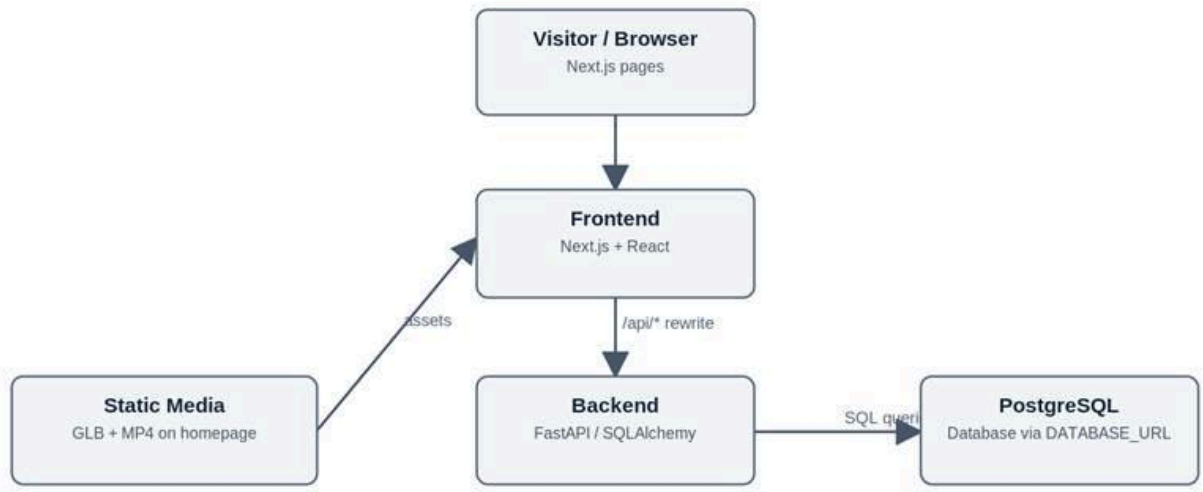


Figure 4.1. Ascenda component architecture (current source design).

4.2 Database Organization

The following tables and representative columns are documented in the current ORM review. The list does not claim to include all fields or constraints.

Table	Representative columns	Purpose
users	id, email, password_hash, role	User accounts
user_profiles	user_id, goals, experience_level, days_per_week, session_minutes	Fitness preferences
assessments	user_id, answers, score, experience_level	Fitness evaluations
equipment / muscle_groups	id, name	Exercise lookups
exercises	id, wger_id, name, difficulty, description, default_role, default_minutes	Exercise catalog
training_programs	id, name, goal, length_weeks	Programs
workout_sessions	id, program_id, name, position	Program sessions
session_exercises	workout_session_id, exercise_id, role, sets, reps, minutes	Prescriptions
workout_records	id, user_id, performed_at, available_minutes	User workout logs
performance_entries	id, exercise_id, reps, weight_kg, status	Performance
exercise_models	exercise_id, model_url, video_url	Model/video metadata

Association tables are exercise_equipment, exercise_muscles (including is_primary), and user_equipment. Key relationships include User to WorkoutRecord (one-to-many), TrainingProgram to WorkoutSession (one-to-many), WorkoutSession to SessionExercise (one-to-many), and Exercise to Equipment/MuscleGroup (many-to-many).

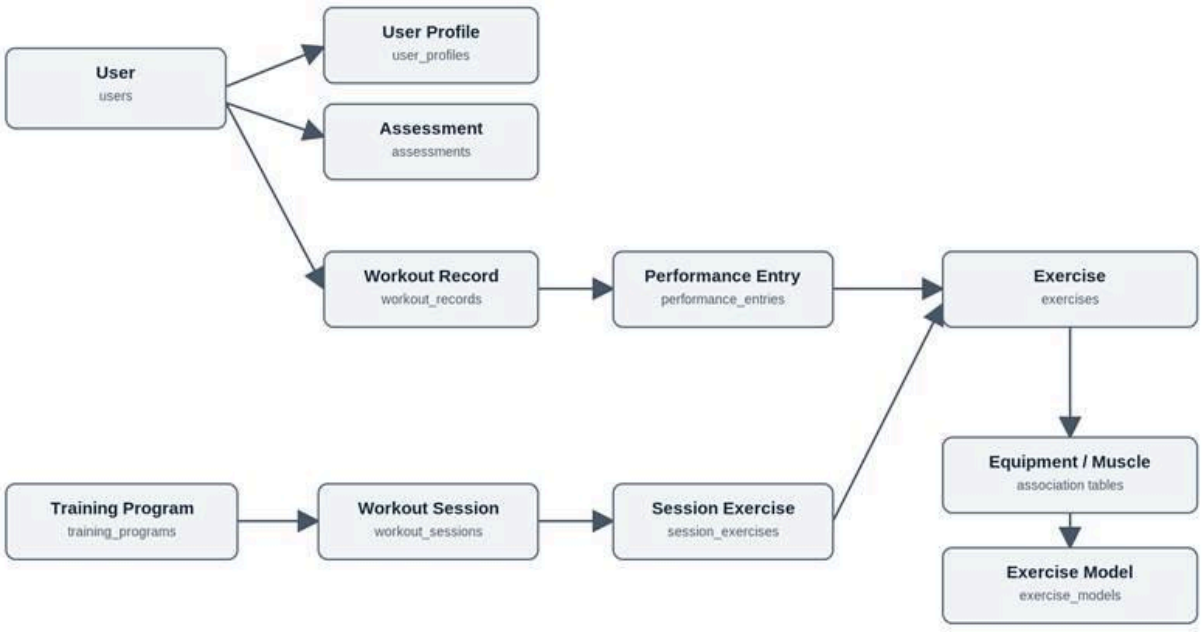


Figure 4.2. Simplified domain/database relationships; not a complete ER diagram.

4.3 Add / Delete / Search Operations

The table describes operations represented by the source review. “Not verified” means an equivalent public API was not identified, not that the database cannot perform the operation.

Data area	Add	Search / display	Update / delete
Users	POST registration creates user	Lookup by email/ID; GET current user	Profile edit and delete not verified
Assessment/profile	ORM models defined	No submission or profile update route verified	Not verified
Exercises / equipment / muscle groups	Import/seed scripts	Filtered exercise search; equipment and muscle lists	Import scripts may update catalog; public write/delete not verified
Programs / sessions	No public create route verified	Program and session filters; session detail	Public update/delete not verified
Workout records / entries	Authenticated POST record and entries	User-scoped history and record detail	Edit/delete not verified
Exercise models	Script attaches metadata	List and detail metadata	Public upload/delete not verified

For example, workout-session search accepts muscle, difficulty, and maximum minutes; matching sessions and prescriptions are retrieved from PostgreSQL. User workout-record requests require bearer authentication and are scoped by user ownership.



Figure 4.3. Workout-session search from browser to database.

4.4 Backend APIs

The FastAPI router uses the /api prefix. These paths are documented in the current backend source review.

Category	Method and endpoint	Function
Health	GET /api/health	Basic liveness check
Authentication	POST /api/auth/register; POST /api/auth/login; GET /api/auth/me	Register, log in, current user
Catalog	GET /api/exercises; GET /api/exercises/{exercise_id}	Search and exercise detail
Lookups	GET /api/equipment; GET /api/muscle-groups	Equipment and muscle options
Programs	GET /api/programs	Filter programs by goal / weeks
Sessions	GET /api/workout-sessions; GET /api/workout-sessions/{session_id}	Search sessions and retrieve exercises
Records	POST /api/workout-records; GET /api/workout-records; GET /api/workout-records/{record_id}	Create and view user workout records
Media	GET /api/exercise-models; GET /api/exercises/{exercise_id}/model	Read media metadata
Query Parameters	muscle, equipment, difficulty, limit, offset, goal, max_weeks	Allows frontend to communicate with backend and return filter requirements

Assessment submission/scoring, profile editing, automatic workout shortening, and personalized recommendations are M1 goals but do not have verified implemented endpoints in the reviewed source.

4.5 Third-Party APIs and Open-Source Components

Component	Role and source-supported status
wger REST API	scripts/import_wger.py imports exercise data to Ascenda PostgreSQL; routine website requests do not call wger live.
React Three Fiber / Drei	Render Scaniverse-exported GLB and hosts a Three.js VideoTexture plane in one interactive 3D scene.
HTML5 video	An HTML video element supplies MP4 to the video texture; playback is embedded inside the 3D Canvas, not a separate side-by-side player.
Next.js / React	Frontend page and UI components.
FastAPI / SQLAlchemy	REST backend, services, repositories, ORM database access.
GitHub Actions / Vercel	Backend-first deployment workflow; successful production run not independently confirmed.

No live Niantic REST call or fully implemented gym scan workflow is established by the reviewed code. The existing model viewer is a separate supporting feature.

4.6 Technology Changes from M0 and M1

M0 lists an initial Google Compute Engine / React / JavaScript / Express / MongoDB stack. M1 changes the planned stack to Vercel, Next.js 16.x / React 19.x, Python 3.14.x / FastAPI, and Supabase PostgreSQL. The current source review follows the M1 architecture, with SQLAlchemy, React Three Fiber, Drei, and wger import scripts added.

Area	M0	M1	M2 reviewed source
Hosting	Google Compute Engine	Vercel	Vercel workflow configured
Frontend	React	Next.js 16.x / React 19.x	Next.js + React
Backend	JavaScript / Express	Python 3.14.x / FastAPI	Python + FastAPI
Database	MongoDB 8	Supabase / PostgreSQL	PostgreSQL via SQLAlchemy
3D	Not specified	Niantic Spatial if used	Static GLB with R3F/Drei

4.7 Deployment and Verification

The repository contains .github/workflows/deploy-vercel.yml. It is configured to deploy the backend first, test /api/health and /api/muscle-groups, then deploy the frontend using BACKEND_URL. The backend database connection is configured through DATABASE_URL.

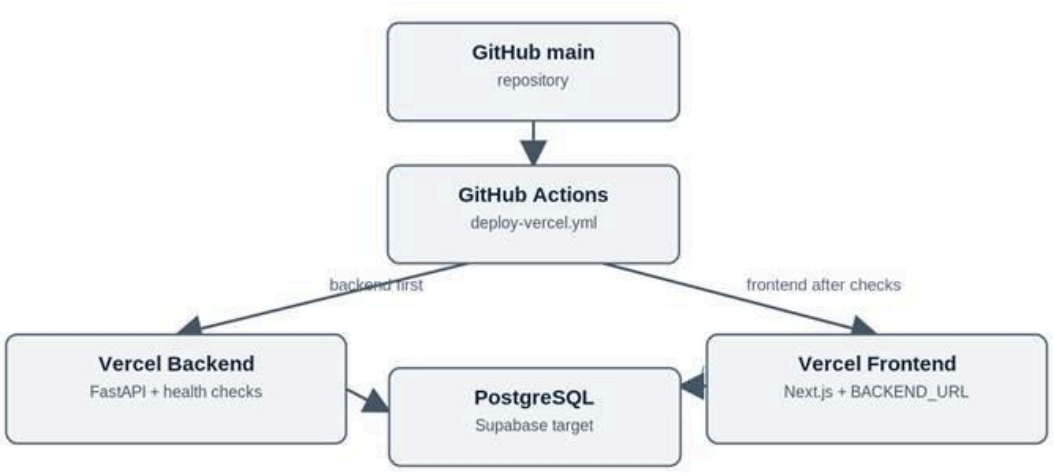


Figure 4.4. Configured deployment path; not proof of production success.

Verification still needed: latest successful workflow run, actual deployed URLs, database connectivity and populated workout-session results. A health response alone does not prove the vertical prototype works from browser to database.

5. High-Level UML Diagrams

This section presents the main UML implementation classes and six sequence diagrams for Ascenda. The diagrams are based on the reviewed source documentation for GitHub main, commit 9139ccd (October 20, 2026).

5.1 High-Level UML Class Diagram

Figure 5.1 shows selected SQLAlchemy ORM classes used for user accounts, exercise catalogs, workout sessions, performance records, and exercise media metadata. It is a high-level class model, not a complete database schema. The attributes match the supplied UML documentation.

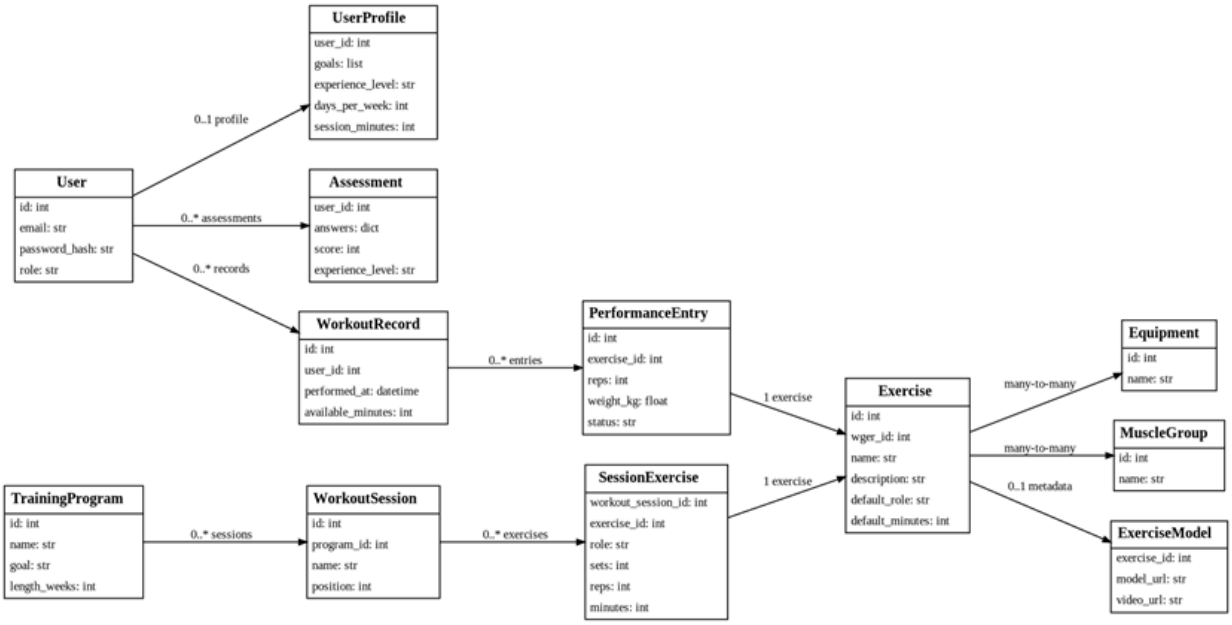


Figure 5.1. Main ORM classes and relationships.

A WorkoutSession contains SessionExercise items that connect to Exercise. SessionExercise stores session-specific role, sets, reps, and minutes. WorkoutRecord and PerformanceEntry store user workout history. Assessment has a database model, but its scoring workflow is not verified.

5.2 High-Level UML Sequence Diagrams

An asterisk (*) in a sequence diagram marks a proposed UI, endpoint, or service. The workflows below represent required user actions and the related architecture at the reviewed source snapshot.

5.2 Registration and Profile Setup

Related requirements: R1, R2. Backend registration implemented; profile UI/editing proposed.

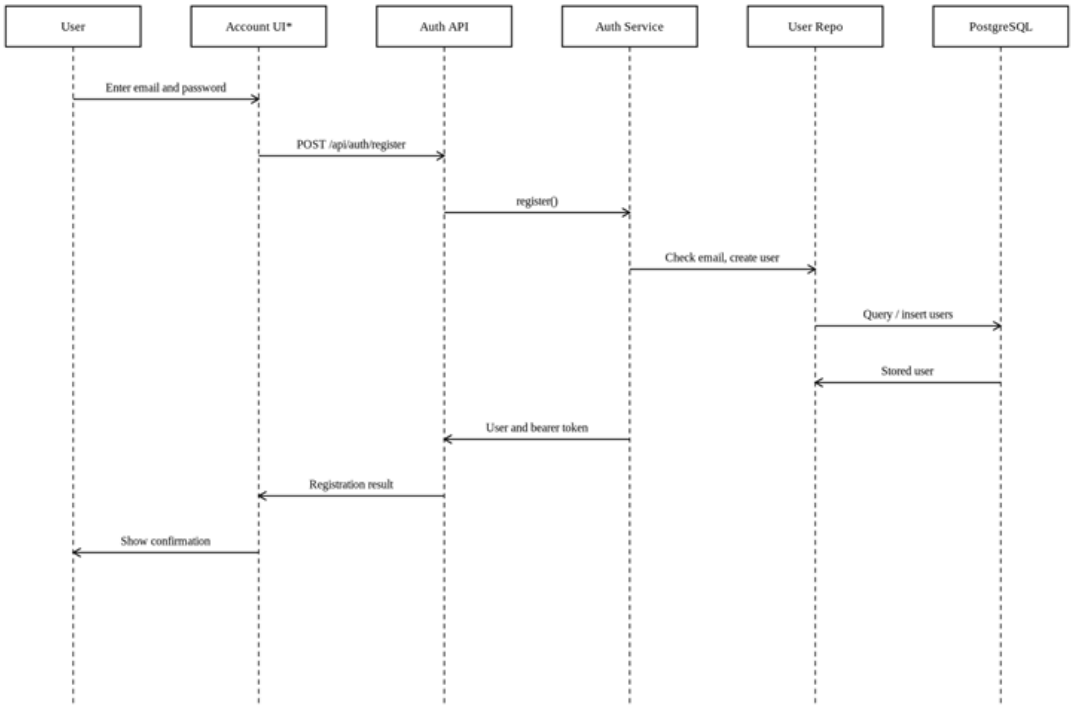


Figure 5.2. Registration and Profile Setup sequence diagram.

The API creates an account and returns a bearer token. Profile editing remains proposed; the UserProfile ORM by itself does not establish a profile editing endpoint.

5.3 Initial Fitness Assessment

Related requirements: R4. Proposed UI, endpoint, and scoring logic; Assessment ORM exists.

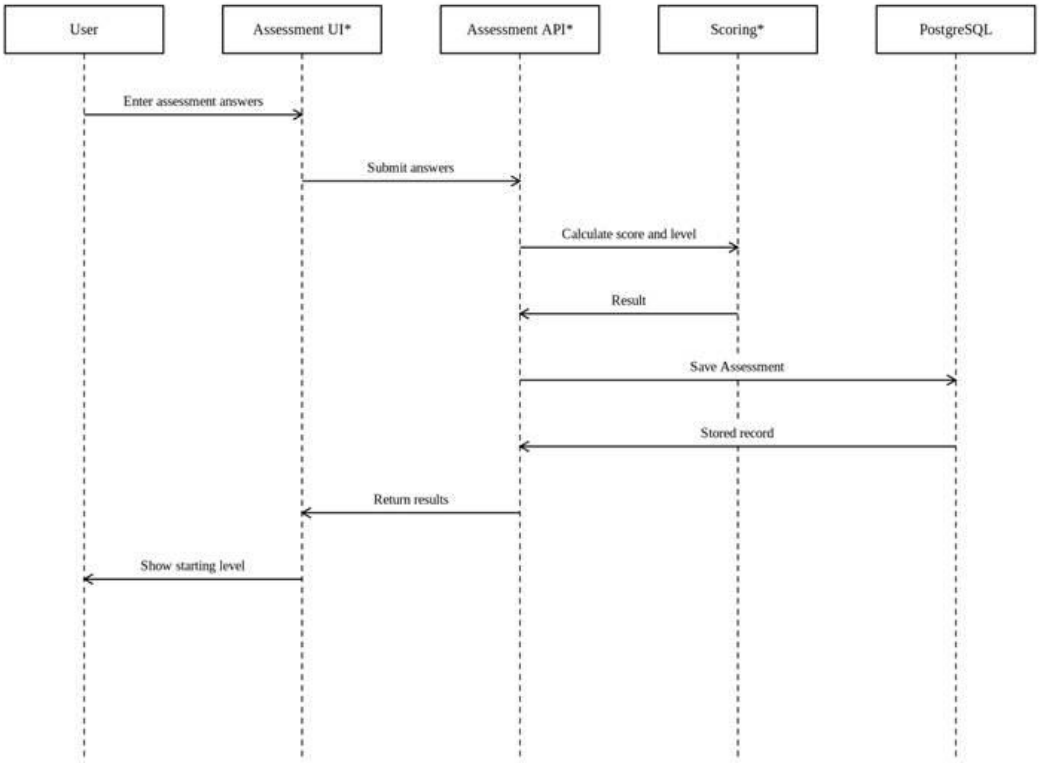


Figure 5.3. Initial Fitness Assessment sequence diagram.

The Assessment model stores answers, score, and experience level. The assessment endpoint and scoring rules are proposed; no implemented assessment service was identified.

5.4 Filter and View Workout Sessions

Related requirements: R3, R5, R6. Frontend and backend source path implemented; live DB results unverified.

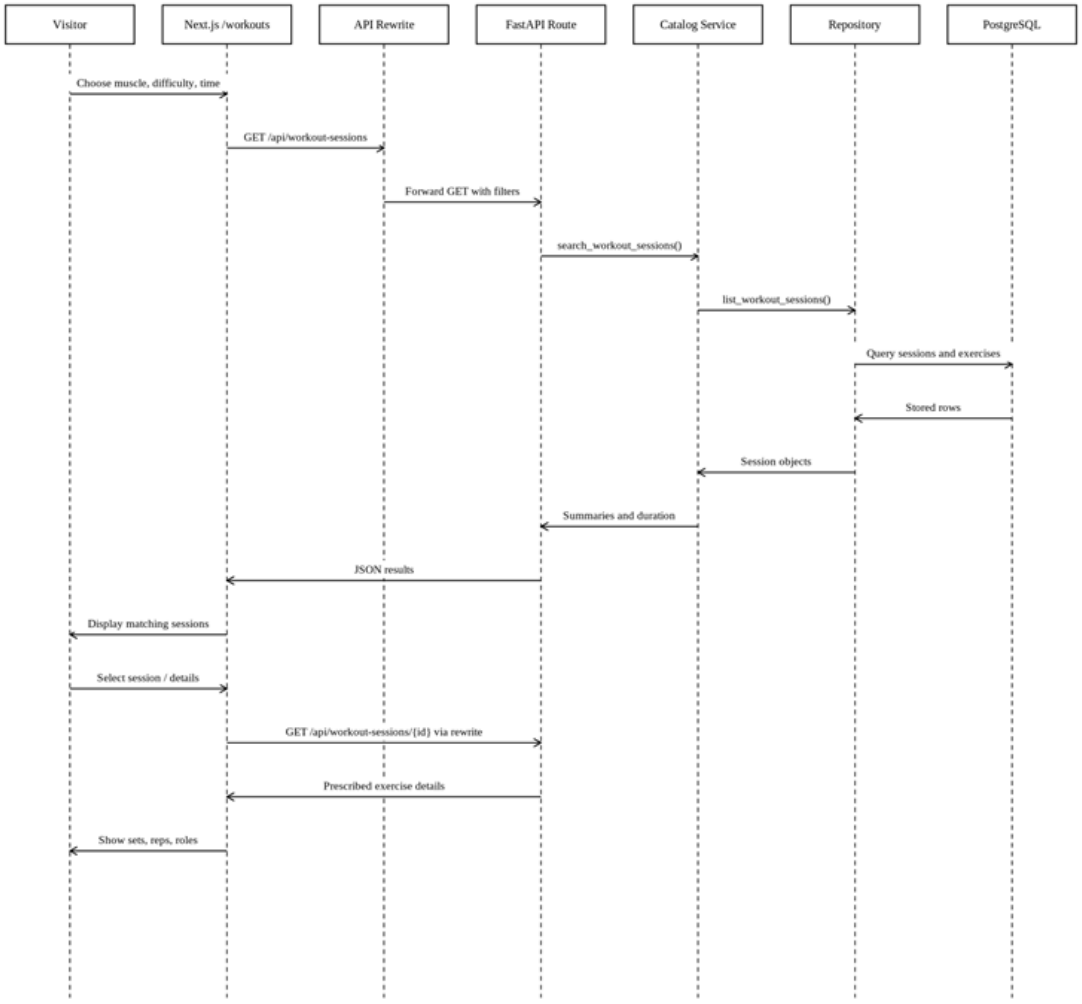


Figure 5.4. Filter and View Workout Sessions sequence diagram.

The /workouts page requests filtered sessions and details from FastAPI through a Next.js rewrite. Source-level integration exists; results from the deployed database have not been independently verified.

5.5 Adapt Workout to Available Time

Related requirements: R6. Proposed adjustment flow; role and duration fields exist.

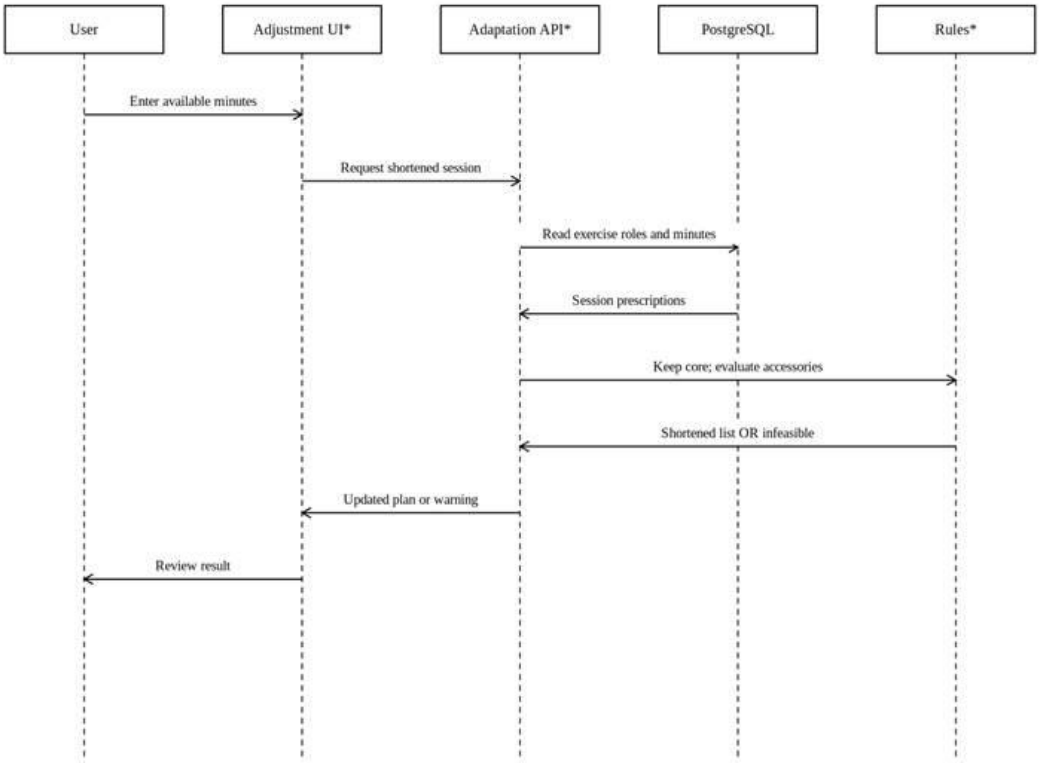


Figure 5.5. Adapt Workout to Available Time sequence diagram.

Workout time adaptation is proposed. Existing session roles and minutes provide useful inputs, but no verified adaptation endpoint or automatic removal algorithm exists.

5.6 Log Workout and View History

Related requirements: R8, R9. Backend routes implemented; user-facing client proposed.

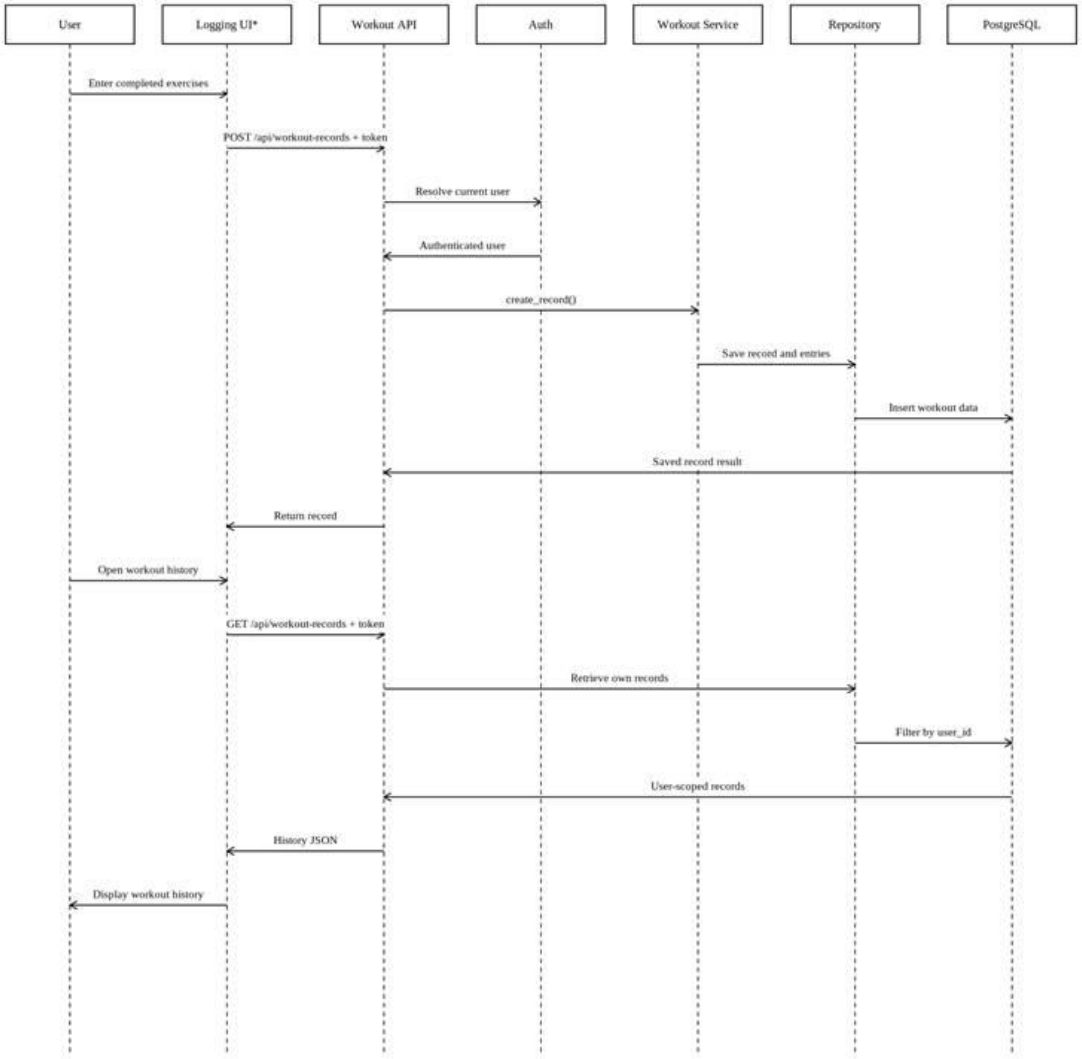


Figure 5.6. Log Workout and View History sequence diagram.

Backend authentication, record creation, and user-scoped history retrieval are implemented. The full browser logging and history screens are not verified.

5.7 View Exercise Demonstration

Related requirements: R3. Homepage GLB and integrated MP4 playback implemented.

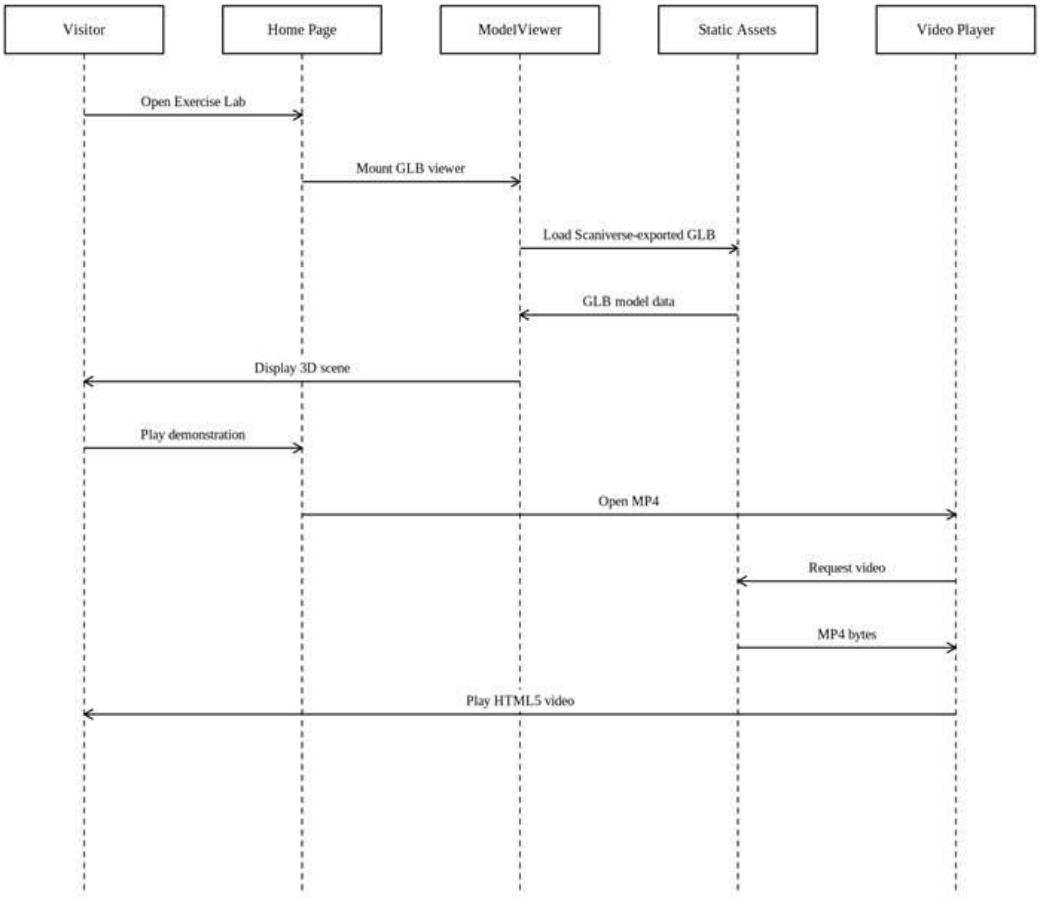


Figure 5.7. View Exercise Demonstration sequence diagram.

The homepage mounts ModelViewer, which loads the Scaniverse GLB and reders VideoPlane within the same React Three Fiber Canvas. VideoPlane creates an HTML video and a Three.js VideoTexture, applies it to a plane mesh, and disposes video/texture resources on replacement. Selection switches between two hard-coded MP4 paths. The video loops, and controls allow playback and plane-position adjustments. This does not demonstrate a live Niantic API or a gym-scanning workflow.

5.8 Requirement Traceability and Implementation Status

Diagram	M1 requirement	Source-review status
Registration and Profile Setup	R1, R2	Backend registration implemented; profile UI/editing proposed
Initial Fitness Assessment	R4	Proposed UI, endpoint, and scoring logic; Assessment ORM exists
Filter and View Workout Sessions	R3, R5, R6	Frontend and backend source path implemented; live DB results unverified
Adapt Workout to Available Time	R6	Proposed adjustment flow; role and duration fields exist

Log Workout and View History	R8, R9	Backend routes implemented; user-facing client proposed
View Exercise Demonstration	R3	Homepage GLB and looping MP4 video texture share a 3D canvas.

6. Possible Project Risks and Mitigation Plans

This section identifies the current risks that may affect Ascenda's development. The risks are based on the team's technology study plan, Milestone 1 requirements, and the architecture developed for Milestone 2.

6.1 Technical Skills Risk

Risk: Ascenda uses several technologies, including Next.js, FastAPI, PostgreSQL, SQLAlchemy, and Vercel. Team members need enough experience with these tools to complete the application and integrate its components.

Mitigation: The team developed a four-week technology study plan in Milestone 1, assigning learning tasks based on each member's role. Members will continue practicing their assigned technologies and sharing knowledge during development.

6.2 Frontend and Backend Integration Risk

Risk: The backend contains APIs for authentication, exercises, workout sessions, and workout records. However, not all frontend features are connected to these APIs. Integration problems may affect the completion of major functions.

Mitigation: The team will prioritize connecting the main frontend pages to existing backend endpoints. API responses and database operations will be tested together to identify integration problems early.

6.3 Schedule Risk

Risk: Some important features, including fitness assessments, workout adaptation, and workout logging interfaces, require additional development. Completing these features within the semester may be difficult.

Mitigation: The team will prioritize essential requirements and divide development work into smaller tasks. The Scrum Master will coordinate progress checks, identify delayed tasks, and help the team adjust its schedule when needed.

6.4 Deployment and Database Risk

Risk: Ascenda uses separate frontend and backend deployments on Vercel, connected to PostgreSQL. Incorrect environment variables, deployment configurations, or database connections may prevent the application from working correctly.

Mitigation: The team will test the complete browser-to-backend-to-database connection. GitHub Actions deployment checks and API testing will help identify configuration errors before demonstrations and submissions.

6.5 Teamwork and Communication Risk

Risk: Team members work on different parts of the project, including frontend, backend, deployment, and documentation. Delayed communication or unclear task responsibilities may cause integration problems.

Mitigation: The team will use Scrum meetings and shared task tracking to communicate progress and blockers. The Scrum Master will coordinate task updates, while GitHub will be used to manage code changes and collaboration.

6.6 Third-Party Content and Licensing Risk

Risk: Ascenda uses exercise information imported from wger and includes exercise demonstration media. External content may have licensing, attribution, or usage restrictions.

Mitigation: The team will review the licenses and attribution requirements of third-party exercise data and media before final submission. Content without confirmed permission will not be included in the final application.

6.7 Specification Gap Risk (Assessment and Adaptation Logic)

Risk: The assessment questions, scoring rules, experience-level thresholds, and the workout-shortening algorithm are not yet defined (Sections 1.2.4 and 2.3). Building these features without an agreed specification may lead to rework or inconsistent behavior.

Mitigation: The team will finalize the assessment questions, score ranges, and core/accessory removal rules in a short written specification before implementation begins. The AI Master and Team Lead will review it, and the backend will implement it with unit tests that confirm the same inputs always produce the same results.

6.8 Security and Access Control Risk

Risk: Ascenda stores personal fitness data and passwords. The administrator permission system has not been verified (Section 1.2.1), and an access-control mistake could expose one user's workout history to another user.

Mitigation: The team will confirm that every user-specific endpoint checks the authenticated user's ownership, keep password hashing and bearer-token authentication, and store secrets such as DATABASE_URL only in environment variables, never in the repository. Role checks will be tested before any administrator features are added.

6.9 Privacy and Consent Risk (Demonstration Media)

Risk: The planned 3D demonstration videos feature a person performing exercises. Using the video of an identifiable person without clear consent could create privacy or legal problems.

Mitigation: The team will only use footage of team members or volunteers who have agreed to its use, record that consent, and avoid showing identifying details where possible. Media without confirmed consent will be removed before submission.

7. Project Management

Ascenda is developed by six team members with assigned responsibilities: Team Lead, Scrum Master, Git Master, AI Master, Backend Lead, and Frontend Lead. These roles help organize the development process and divide tasks based on each member's responsibilities.

The team established a four-week technology study plan in Milestone 1. The plan includes frontend development with Next.js, backend development with FastAPI and PostgreSQL, deployment with GitHub Actions and Vercel, and research for additional application features.

The Scrum Master is responsible for coordinating Scrum meetings, tracking task progress, and identifying blockers. Team members share their completed work, current tasks, and technical difficulties during progress discussions. Meeting minutes are shared with the team to keep members informed about responsibilities and project updates.

The Product Manager conducts biweekly one on one check-in with each team member in addition to team meetings twice a week. This helps each team member to share their biweekly updates, any issues that they are facing, their coordination with other team members, ideas they

want to implement, and exchange views with the Product Manager. The Product manager creates the meeting notes and lists out tasks to be completed before the next one on one meeting. The notes are stored in the shared team's google drive.

GitHub is used for source code management and collaboration. The team also planned to use GitHub Projects for task assignments and progress tracking. Development tasks are divided between frontend, backend, deployment, and supporting features.

For Milestone 2, the main project management focus is coordinating the architecture design, UI mockups, UML diagrams, and vertical software prototype. Clear task ownership and regular communication help the team identify integration problems and manage the remaining development work.

7.1 M2 Task Coordination and Ownership

The team established roles: Alisha Sinha (Product Manager), Ian Brown (Scrum Master), Zaw Htut (Git Master), Prashrit Magar (AI Master), Geo Wong (Backend Lead), and (Peter) Dong Wu Yu (Frontend lead). The repository shows collaborative contributions through feature branches, pull requests, and merges to main. The code is merged to the main brain by GitHub Master. Repository records provide evidence of code integration and meeting attention and Scrum discussion outcomes are handled via a blend of Google Doc, Discord, Zoom, and Notion.

7.2 Sprint Tracking and Transparency

The team holds an out-of-session meeting via Zoom every Sunday at 2:00PM. The team reviews sprint tasks documented in Notion and updates tasks based on the assigned member's report. Chronologically stored comments are made to address changes or updates made to the planning of sprint tasks during meeting discussion and agreed upon by the prior to meeting conclusion.

7.3 M2 Implementation and Integration Process

The project separates frontend development under app/frontend, backend services under app/backend, and milestone documentation under milestones/m2. GitHub feature branches and pull requests have been merged into main. The deploy-vercel.yml workflow deploys the backend first, performs basic API checks, and then deploys the frontend with the backend URL.